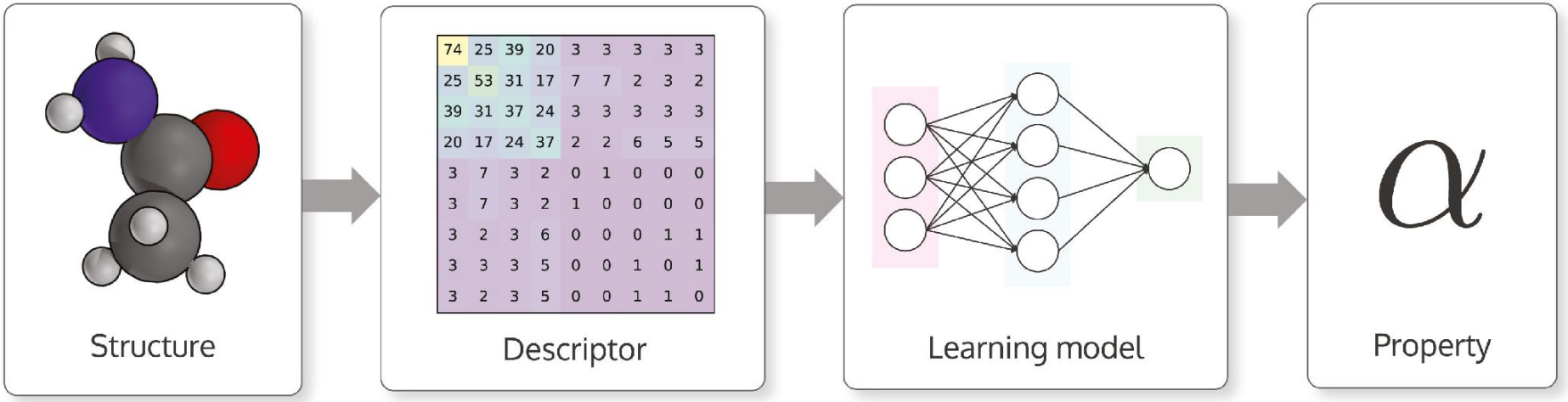
Lecture #8: Graph neural networks for materials science

Slides by Artem Dembitskiy

Goals/Agenda

- Know how to represent an atomic structure for ML
- Learn about fundamentals of graph neural networks (GNNs)
- Crystal graph convolutional neural network overview

Typical Materials Informatics workflow

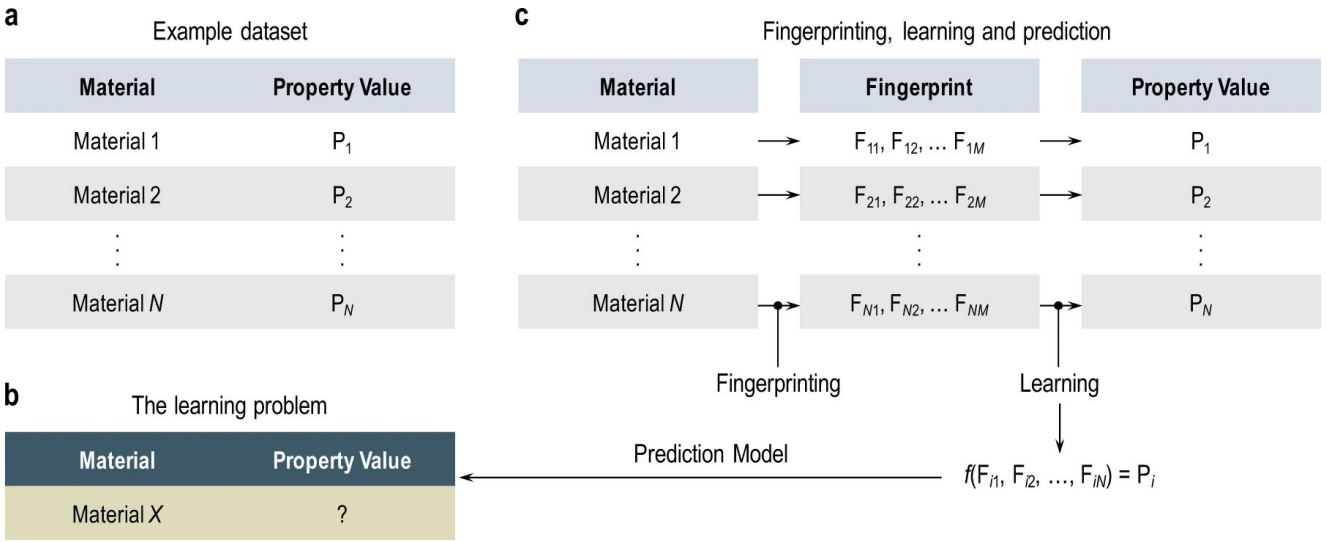


How to represent an atomic structure in a machine readable format?

Previously on

- Aggregation of element features
- Aggregation of structure features
- Feature engineering
- …

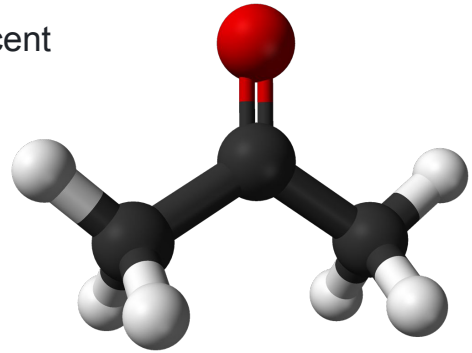
i.e., we use tables to train models

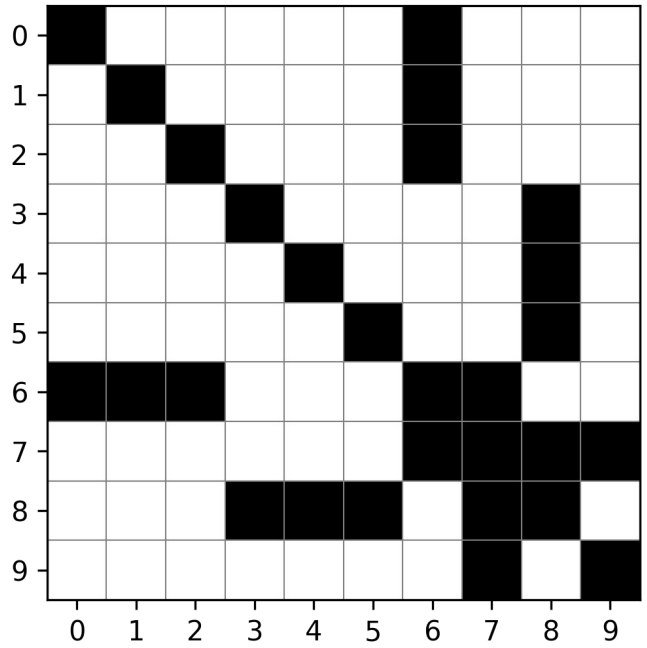


What if we could give a machine-learning model the atomic structure itself and let it learn which structural features matter?

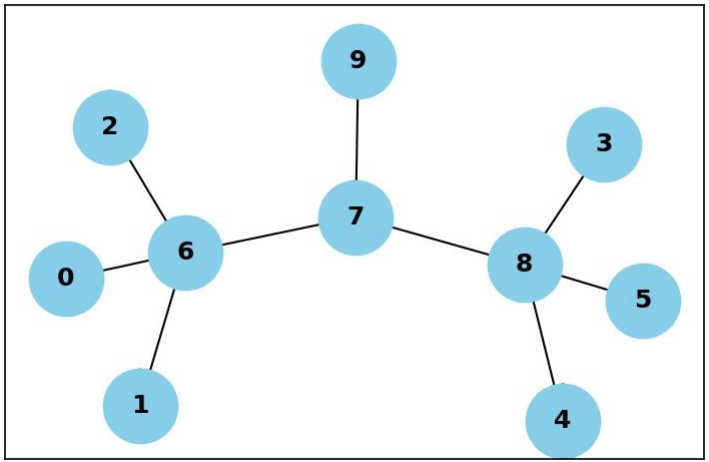
Adjacency matrix

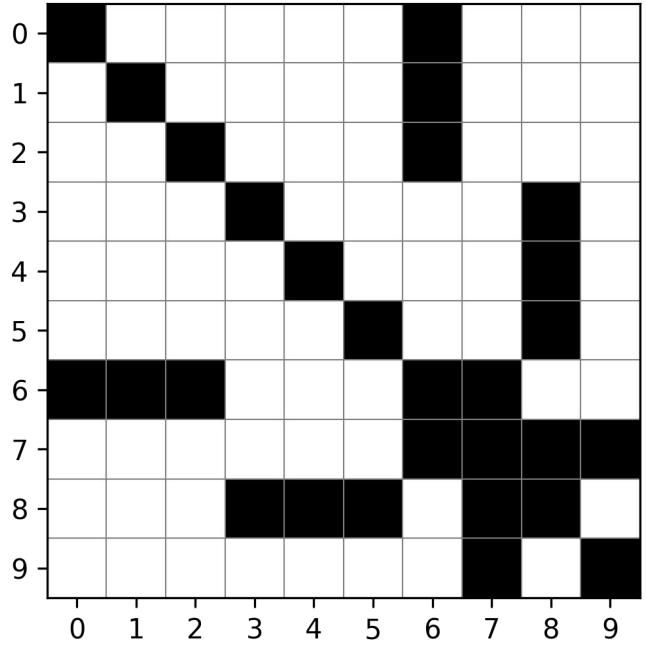
- Encodes graph connectivity (adjacent pairs)
- Represents a finite graph
- Self-loop are usually included
- Sparse representation
  - A lot of zeros
  - N atoms, m=average num of neighbours -> $N_{zero} = N^2 - mN$
  - hence edge list is used





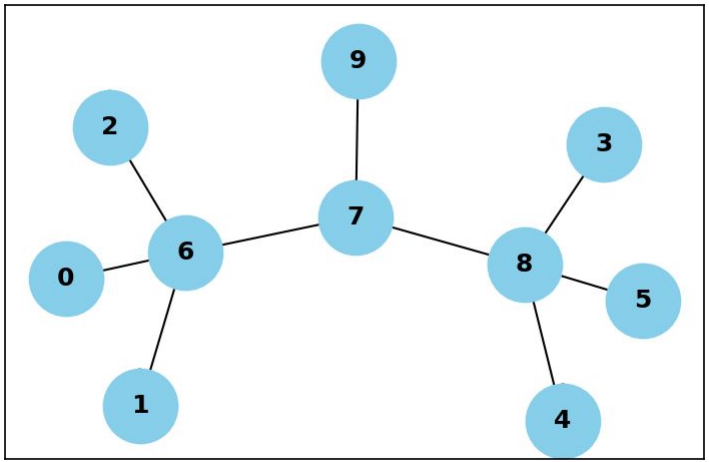
These are equivalent but different representation





Graph representation

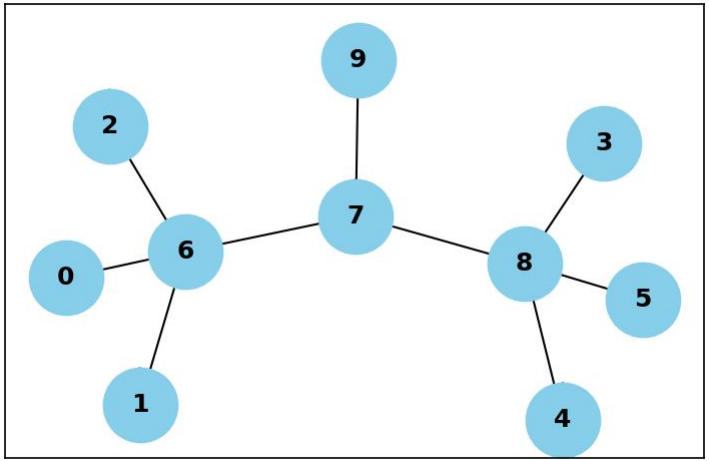
- Nodes = Atoms
- Edges = Bonds
- G = (*E*, *V*) - graph
  - *V* - set of nodes v_i
  - *E* - set of edges $\{e_{ij}\}$ between the nodes



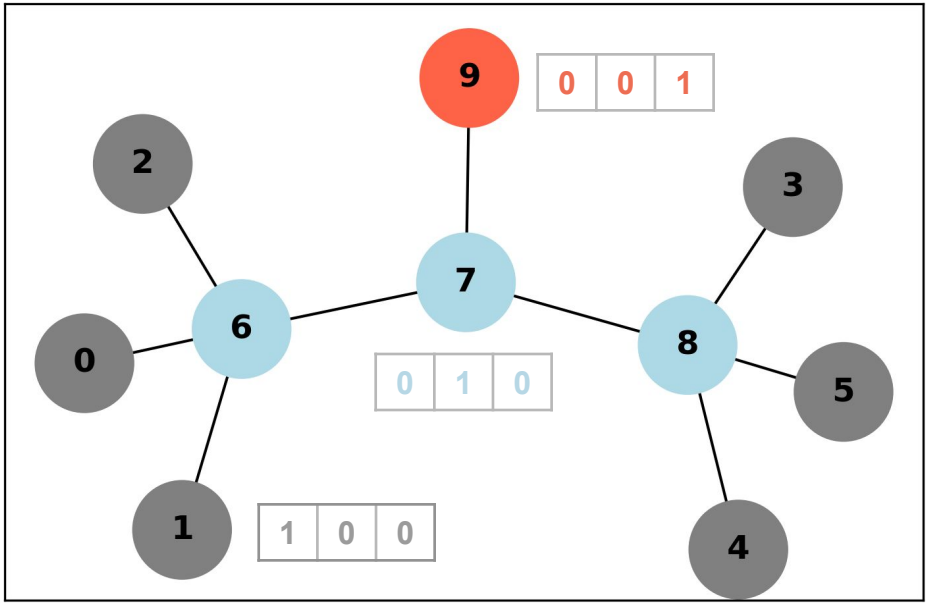
Graph representation

We convert coordinates and atomic numbers {$\boldsymbol{R}$, Z} into graph (E, V)

How to distinguish different atoms?



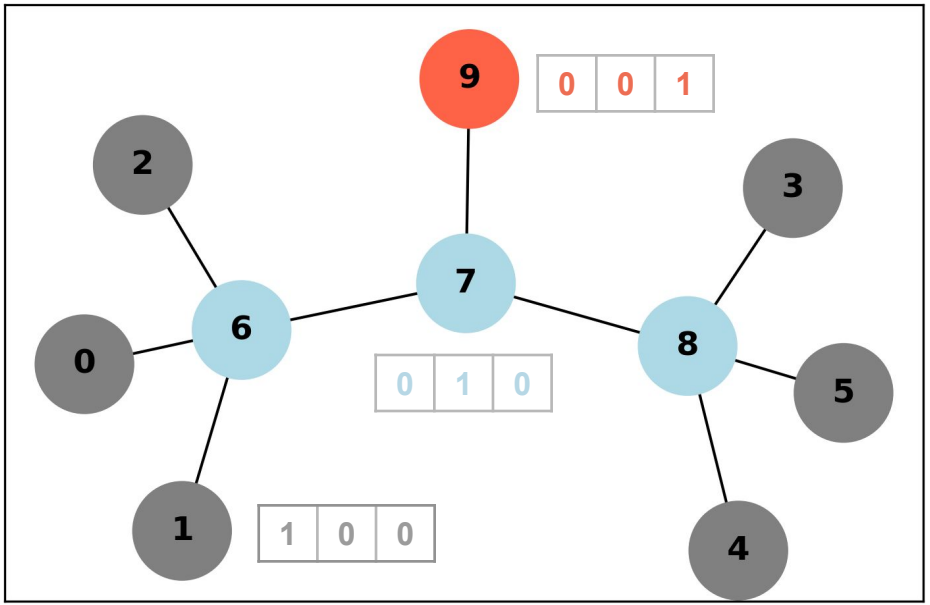
Hydrogen	1	0	0
Carbon	0	1	0
Oxygen	0	0	1



Node encoding

- Naive way - scalar features
  - atomic weight
  - ionic radii
  - number of valence electrons, etc
- Alternative (commonly used) way
  - One-Hot encoding
  - Embeddings

Hydrogen	1	0	0
Carbon	0	1	0
Oxygen	0	0	1



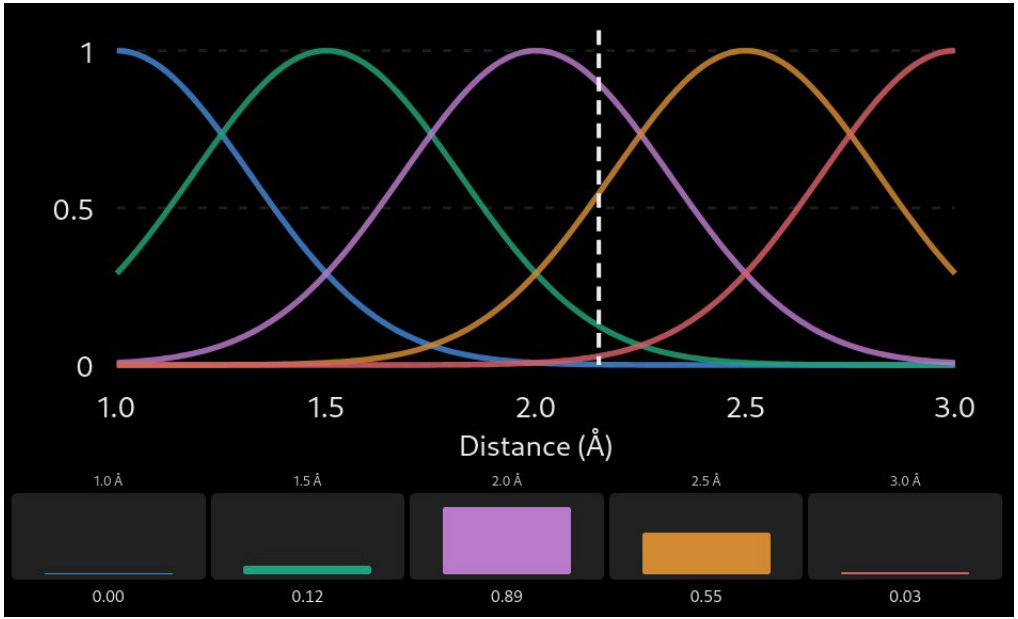
Graph representation

We convert coordinates and atomic numbers {***R***, *Z*} into graph (*E*, *V*)

How to distinguish different bonds?

Edge encoding

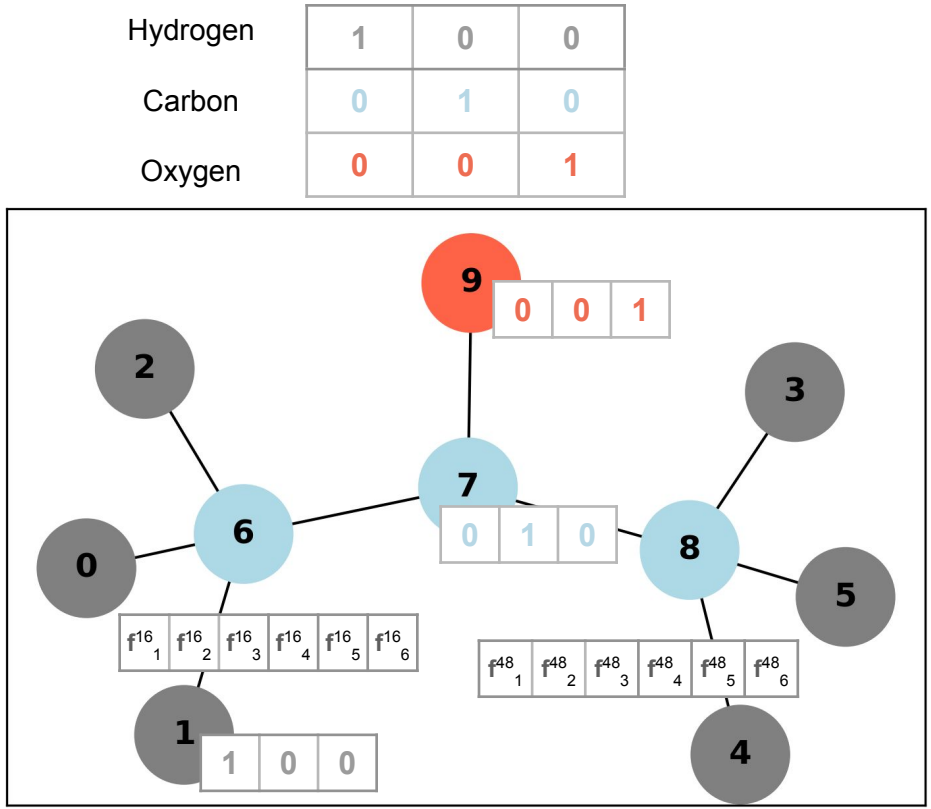
- Naive way - scalar features
  - Bond length
  - Bond ionicity
- More ML convenient way
  - Bond distance expansion
    - e.g. gaussian expansion
    - Edge e_{ij} feature = $[f_1^{ij}, f_2^{ij}, \ldots, f_n^{ij}]$



$$f_i = \exp(\frac{-(d-\mu_i)^2}{2\sigma^2})$$

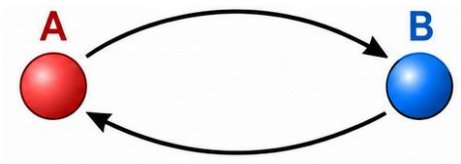
Graph representation

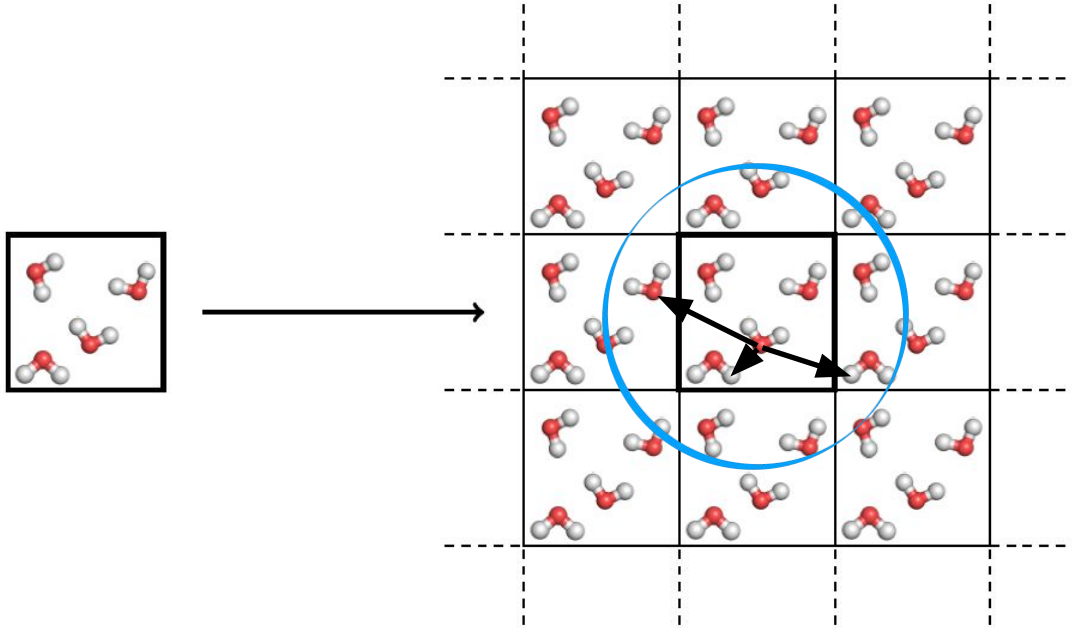
- We convert coordinates and atomic numbers {***R***, *Z*} into graph (*E*, *V*)
- Where each node and edge have their attributes (features)
- Note:
  - There are two edge directions: {i,j} and {j,i}
  - If direction matters, the graph is directed and e_{ij} != e_{ji}



How to deal with PBC in crystals?

- Consider periodic images of the unit cell within the cutoff radius.
- For each atom in the central cell, collect all neighbors within r_{cut}, including periodic images.
- Allow multiple edges between the same pair of nodes to represent different periodic images.

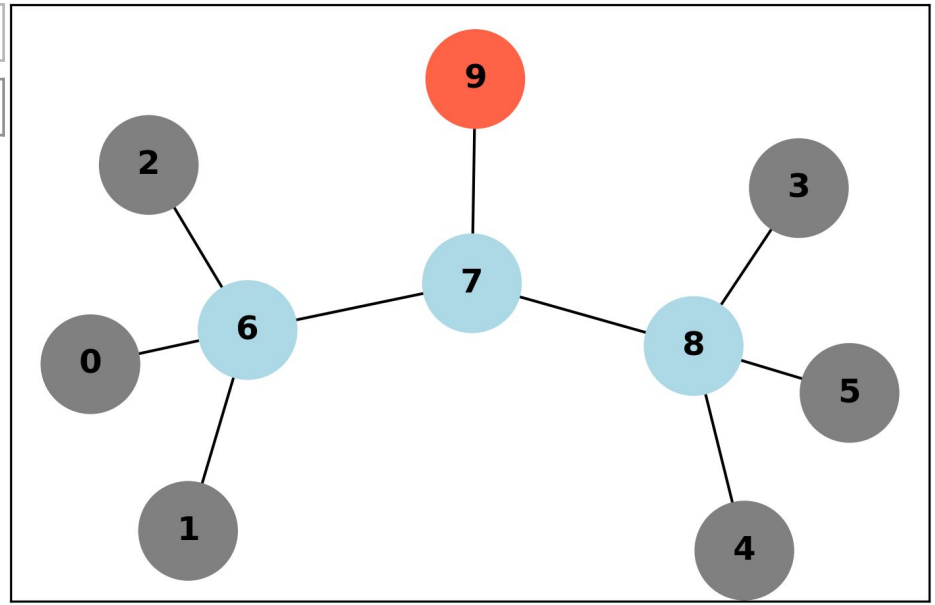




How to get predictions from graph:

Message passing

0	0	1
0	1	0
1	0	0



Message passing - main idea of GNN

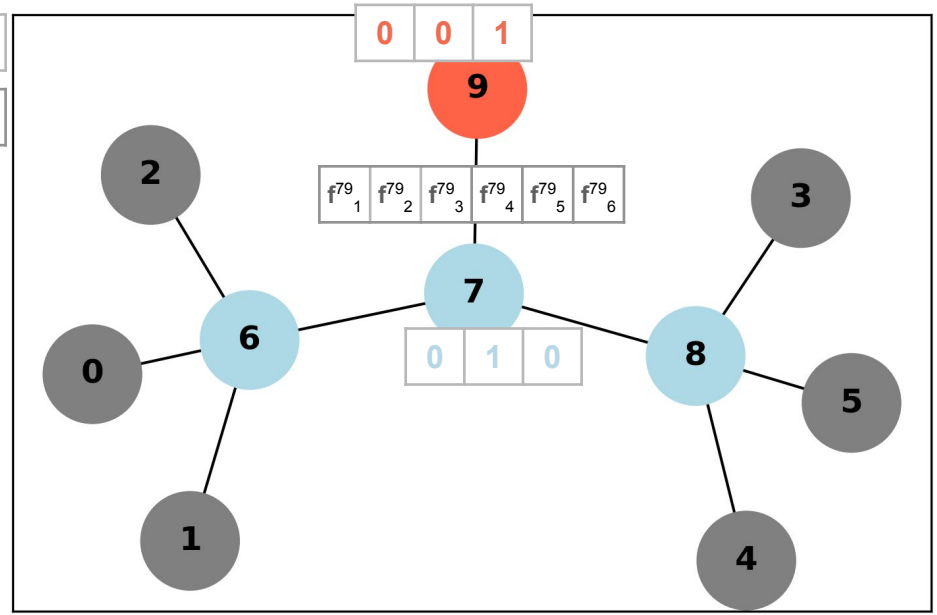
- We want neighbours to exchange the information
- For that we sent message between them
- And update state of each node

0	0	1
0	1	0
1	0	0

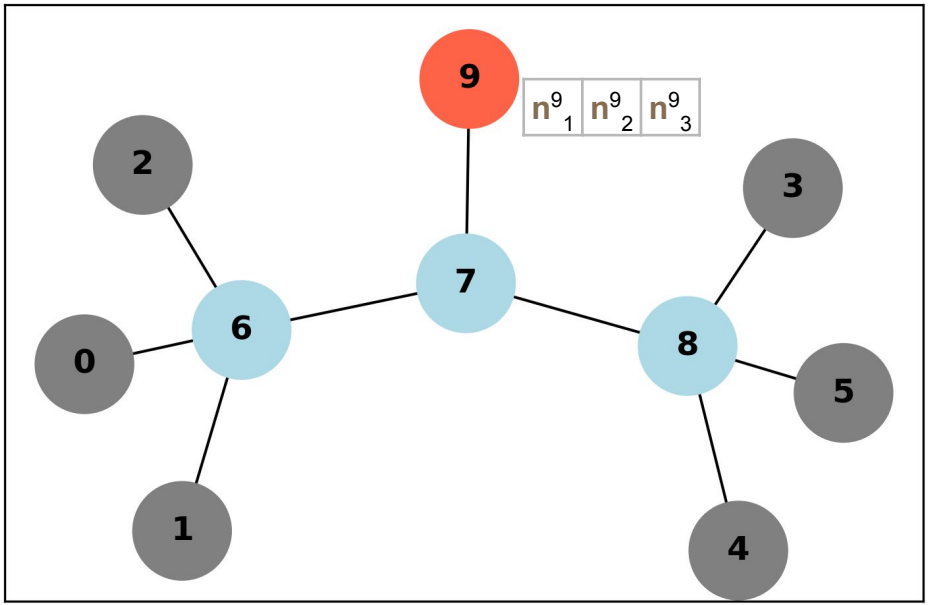
Message from 7th to 9th node

- Concatenation of node and edge features

m = | 0 | 1 | 0 | f^{79}_1 | f^{79}_2 | f^{79}_3 | f^{79}_4 | f^{79}_5 | f^{79}_6 |



0	0	1
0	1	0
1	0	0



9th node update

- Input vector: concatenated source (9), target (7), and edge features

0	0	1	+=	MLP(	0	0	1	0	1	0	f^{79}_1 f^{79}_2 f^{79}_3 f^{79}_4 f^{79}_5 f^{79}_6	)

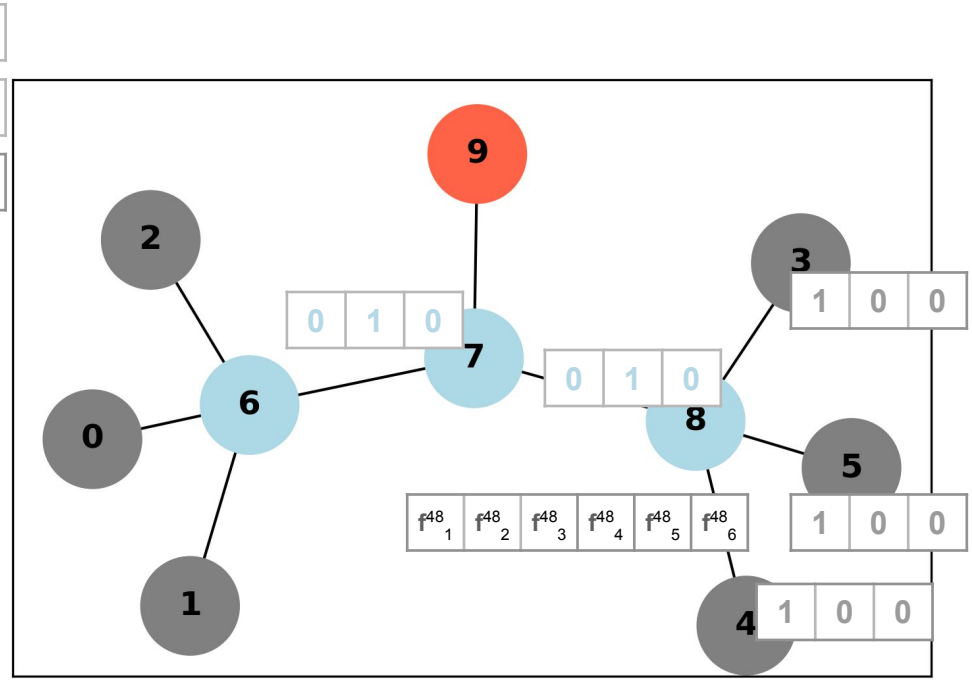
9th node new state

n^9_1	n^9_2	n^9_3

MLP - Multilayer Perceptron - nonlinear function that can be trained:
e.g $m_{79} = W_2$ ReLU $(W_1 z_{79} + b_1) + b_2$, W and b are training parameters
ReLU(x) = max(0,x) - nonlinear activation function

Message from 8th node neighbours

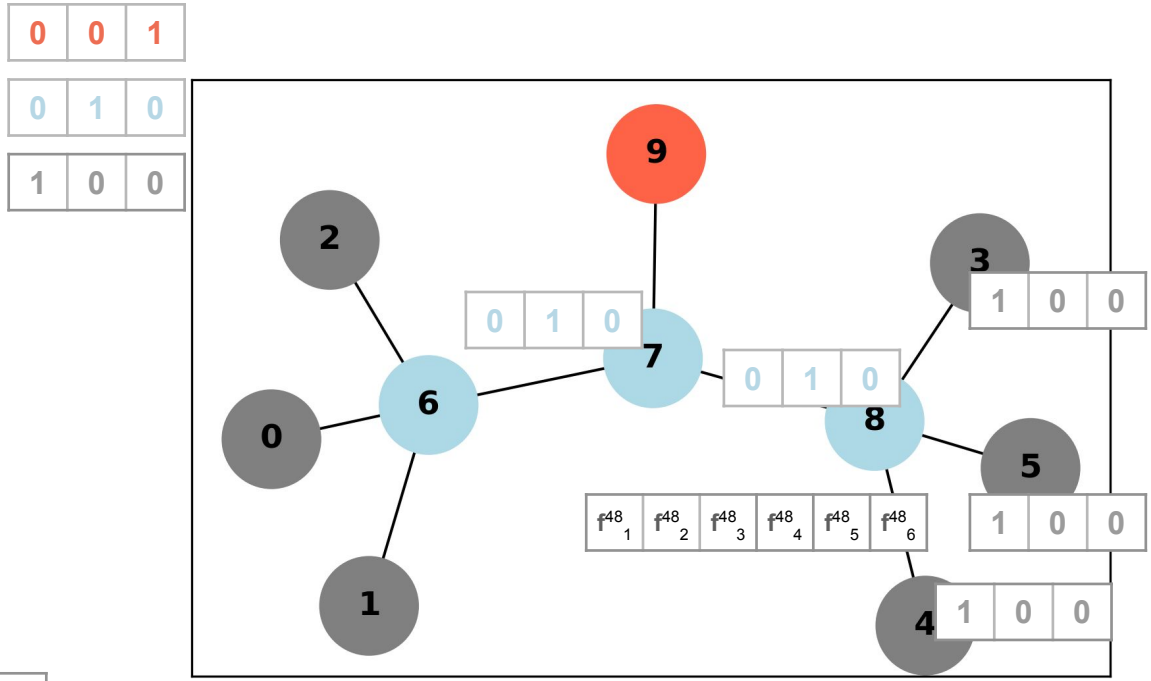
- Concatenation of node and edge features
- Summation (or averaging) of the per-pair messages



$$m = \begin{bmatrix} 1 & 0 & 0 & f^{48}_1 & f^{48}_2 & f^{48}_3 & f^{48}_4 & f^{48}_5 & f^{48}_6 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & f^{58}_1 & f^{58}_2 & f^{58}_3 & f^{58}_4 & f^{58}_5 & f^{58}_6 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & f^{38}_1 & f^{38}_2 & f^{38}_3 & f^{38}_4 & f^{38}_5 & f^{38}_6 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 & f^{78}_1 & f^{78}_2 & f^{78}_3 & f^{78}_4 & f^{78}_5 & f^{78}_6 \end{bmatrix}$$

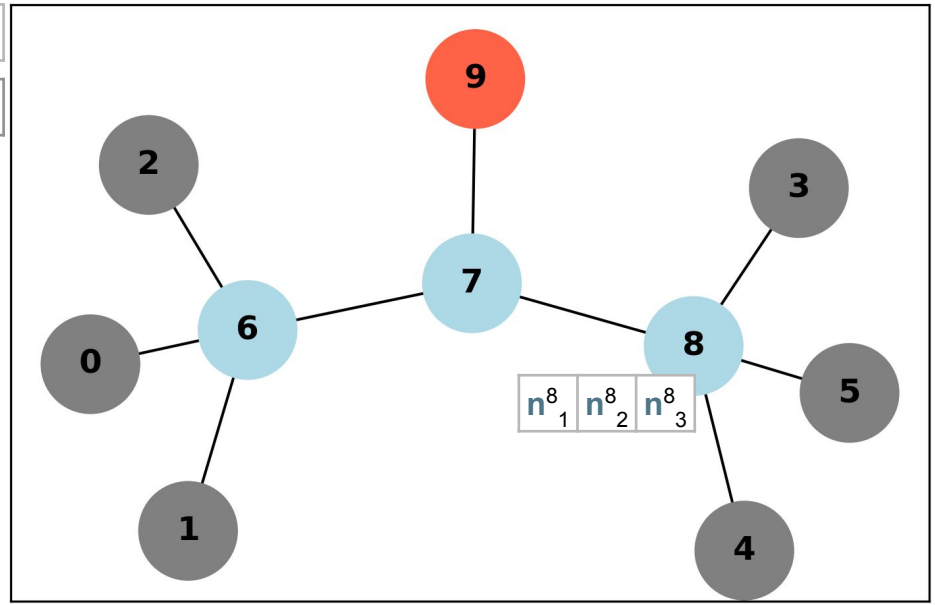
Message from 8th node neighbours

- Concatenation of node and edge features
- Summation (or averaging) of the per-pair messages



m = | 3 | 1 | 0 | f_1 | f_2 | f_3 | f_4 | f_5 | f_6 |

0	0	1
0	1	0
1	0	0



8th node update

0	1	0

+= MLP(

3	1	0	0	1	0	f_1	f_2	f_3	f_4	f_5	f_6

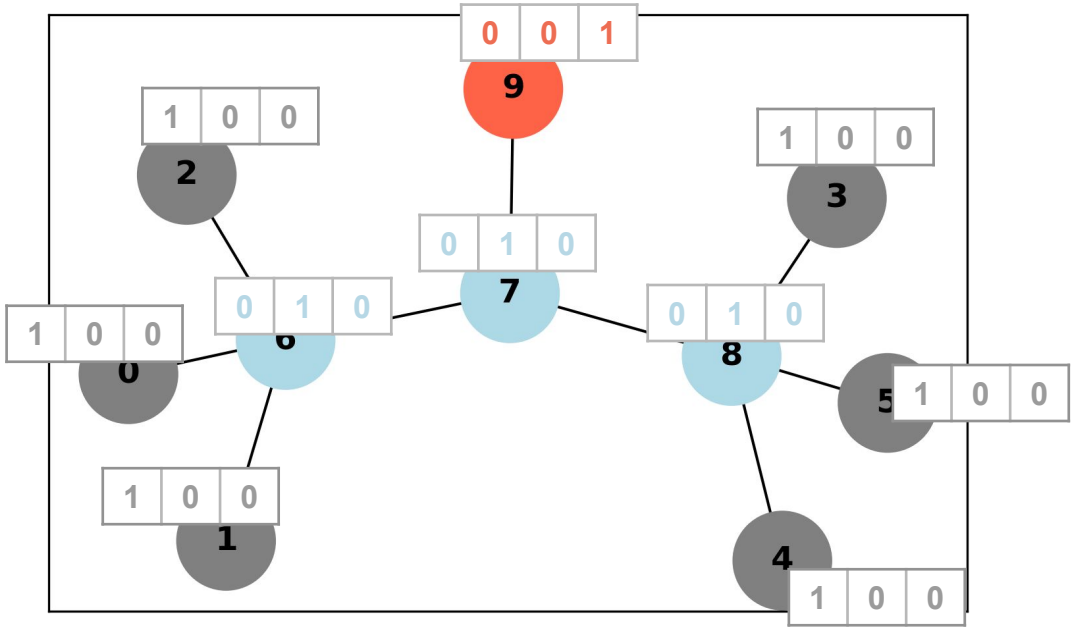
)

8th node new state

n^8_1	n^8_2	n^8_3

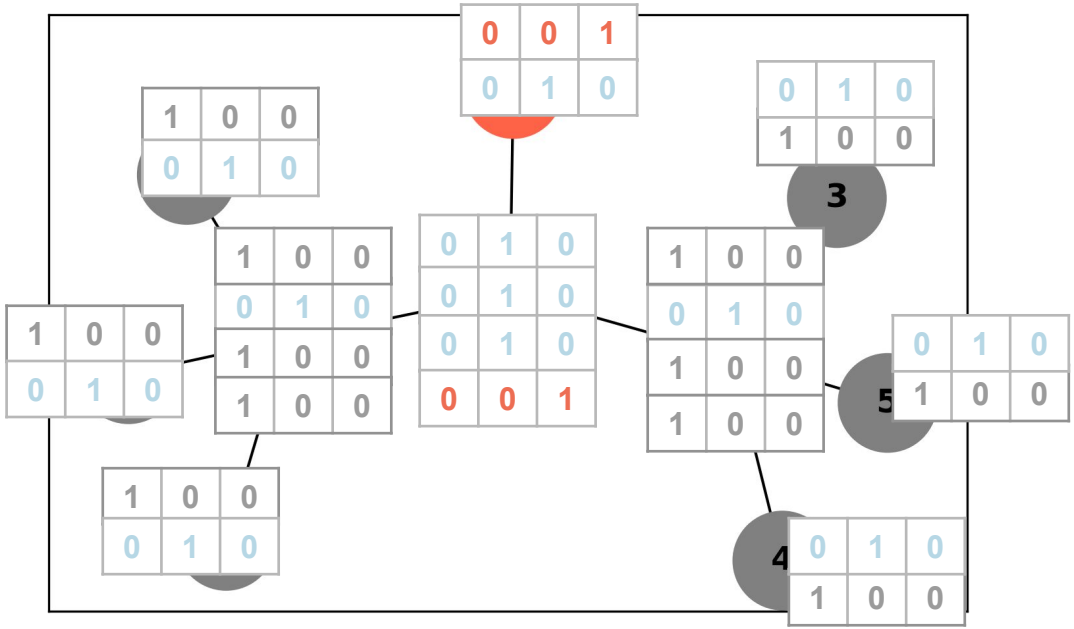
Before massage passing

Note: edge features are not shown

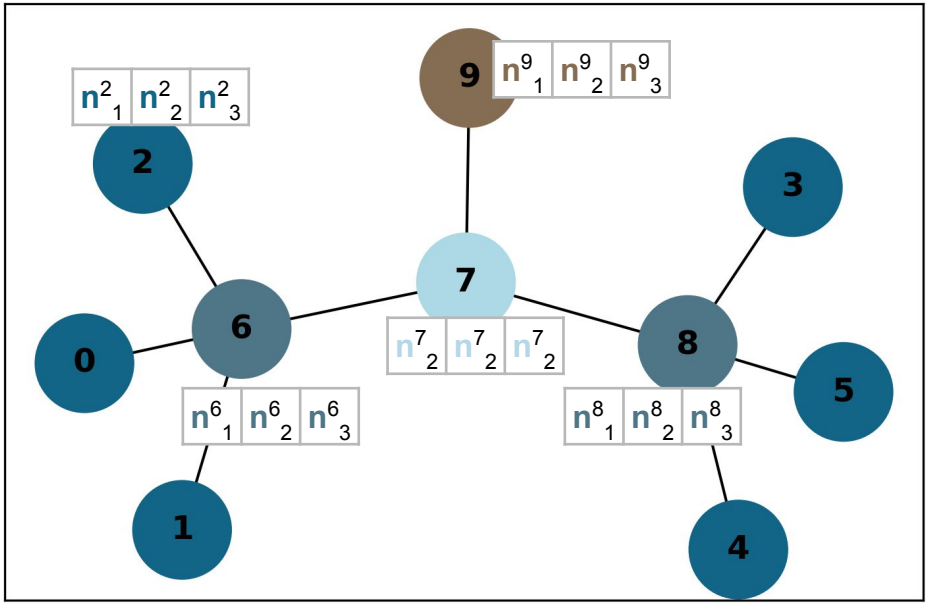


After message passing

Note: edge features are not shown



After message passing

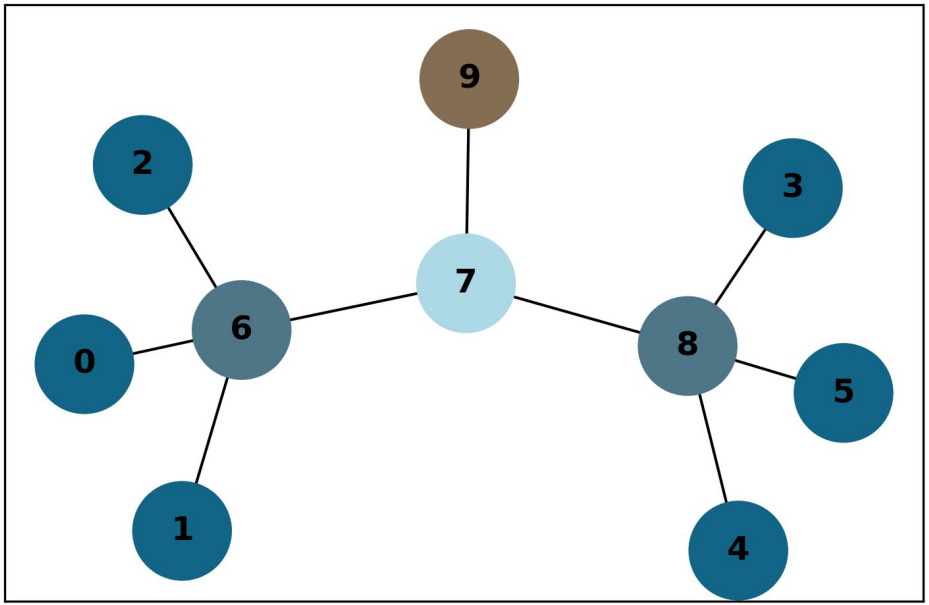


How to get a single value prediction for the entire graph?

Global pooling (readout) layer

- Pool updated node attributes into a single vector
- Using permutation invariant operation (e.g., min, max, mean, sum)

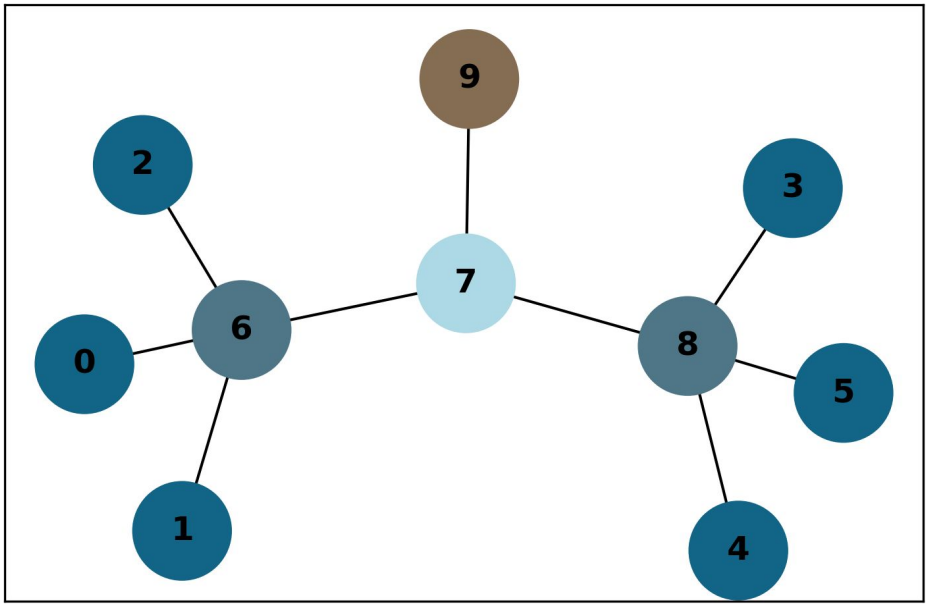
$$\begin{bmatrix} n^1_1 & n^1_2 & n^1_3 \end{bmatrix} + \dots + \begin{bmatrix} n^9_1 & n^9_2 & n^9_3 \end{bmatrix}$$



Global pooling (readout) layer

- Pool updated node attributes into a single vector
- Using permutation invariant operation (e.g., min, max, mean, sum)

h_1	h_2	h_3



Permutation invariance?

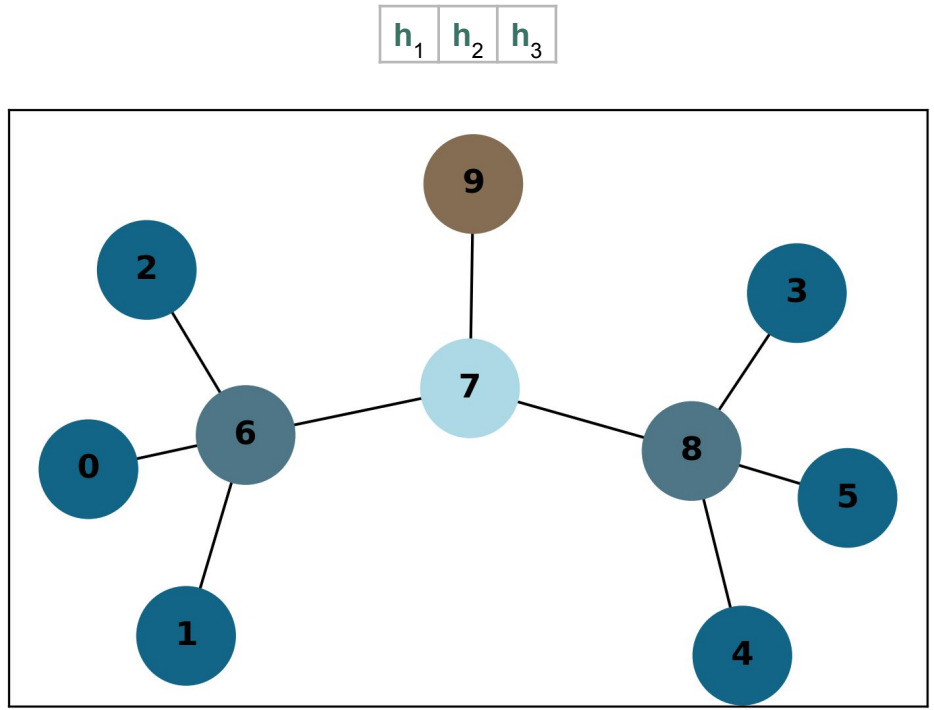
- Change in order of atoms should not affect the model prediction

We also want translation and rotation invariance

- Rotating or translating atomic position should not affect the model prediction

Final prediction

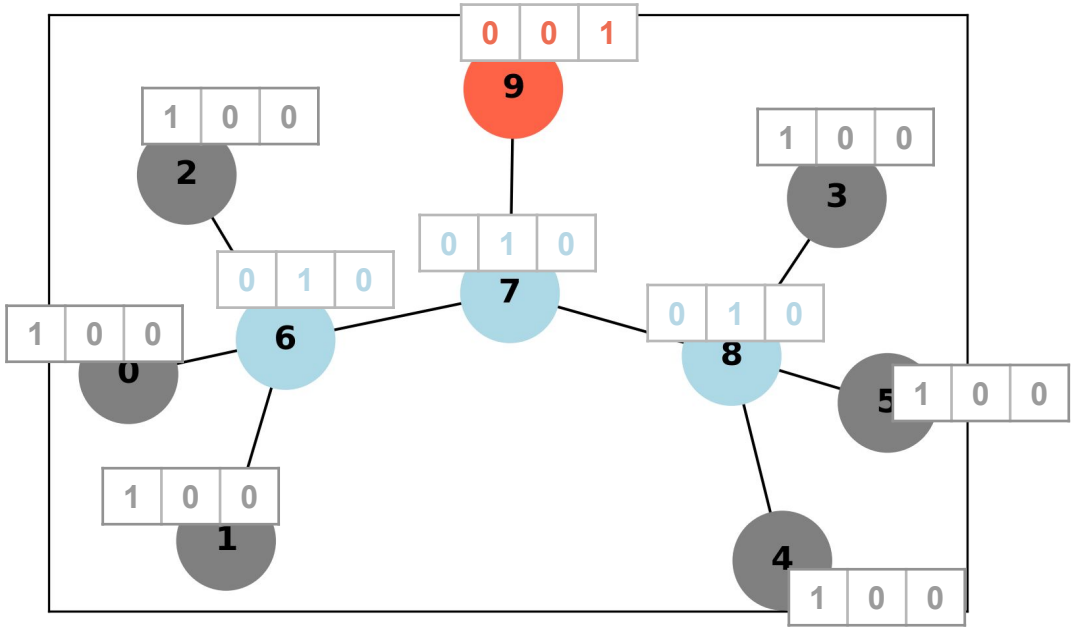
- MLP(h_1 h_2 h_3)
  - e.g., MLP(3 -> 16 -> 1)
  - 3 - input vector size
  - 16 - hidden layer size
  - 1 - output size



Receptive field: how GNN “see” long-range surrounding

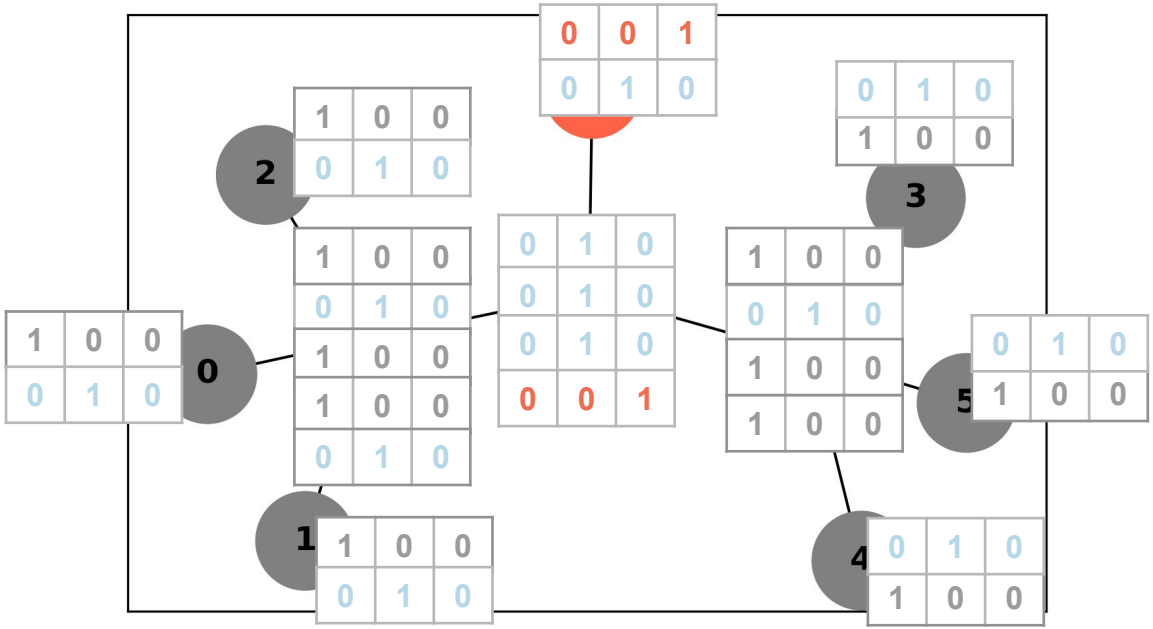
In the example

- We updated node attributes only ones
- In principle we can do it unlimited number of times



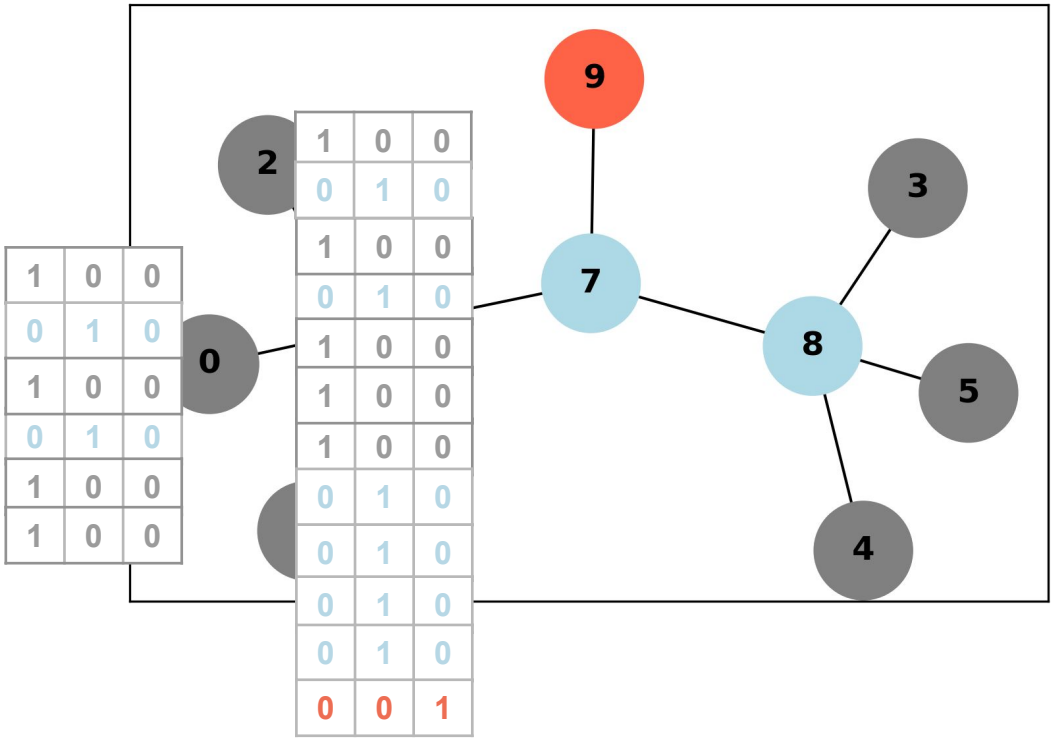
Consider 0th node

- After the first update
  - It knows about 6th node



Consider 0th node

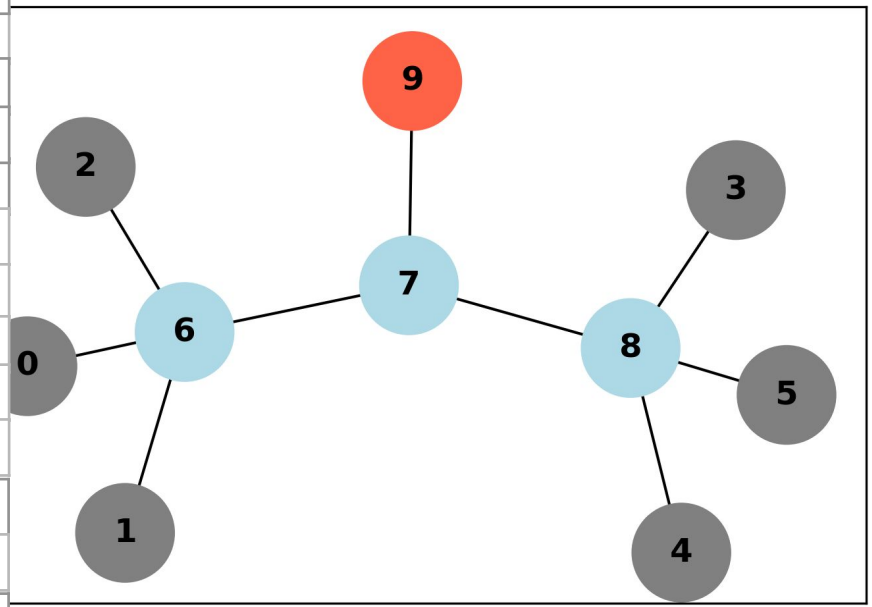
- After the first update
  - It knows about 6th node
- After the second update
  - It knows about 1th, 2th, and 7th nodes
  - 7th node contains information about 9th node



Consider 0th node

- After the first update
  - It knows about 6th node
- After the second update
  - It knows about 6th, 1th, 2th, and 7th nodes
- After the third update
  - It knows about 6th, 1th, 2th, 7th, 8th, and 9th nodes

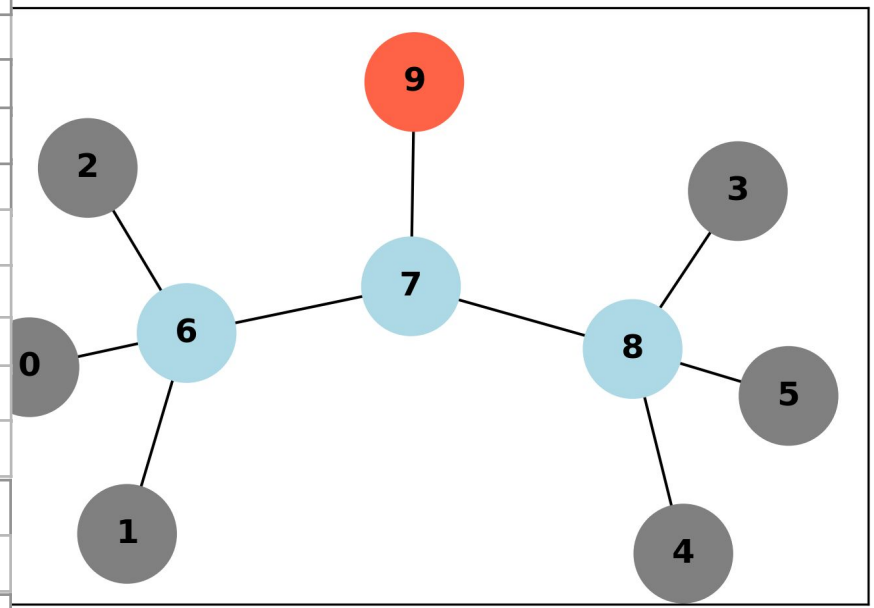
1	0	0
0	1	0
1	0	0
0	1	0
1	0	0
1	0	0
1	0	0
0	1	0
0	1	0
0	1	0
0	1	0
0	0	1
1	0	0
0	1	0
1	0	0
0	1	0
1	0	0
1	0	0



Consider 0th node

- With each message-passing step, a node's receptive field increases
- First NN -> NN of first NN -> and so on

1	0	0
0	1	0
1	0	0
0	1	0
1	0	0
1	0	0
1	0	0
0	1	0
0	1	0
0	1	0
0	1	0
0	0	1
1	0	0
0	1	0
1	0	0
0	1	0
1	0	0
1	0	0

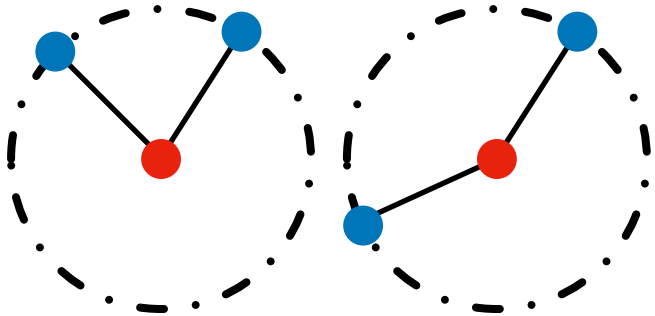


GNNs challenges

- Oversmoothing - many message passing layers
  - All nodes converge to the same average representation
  - The GNN can no longer distinguish local structural roles
- Expressivity limit - different structures may have the same graph
  - Graphs don’t know about angles

Expressivity limit example

- Indistinguishable geometries for the discussed graph representation
- The same energies will be predicted for these configurations of a molecule
- Higher order representations are required



Overcoming GNNs challenges

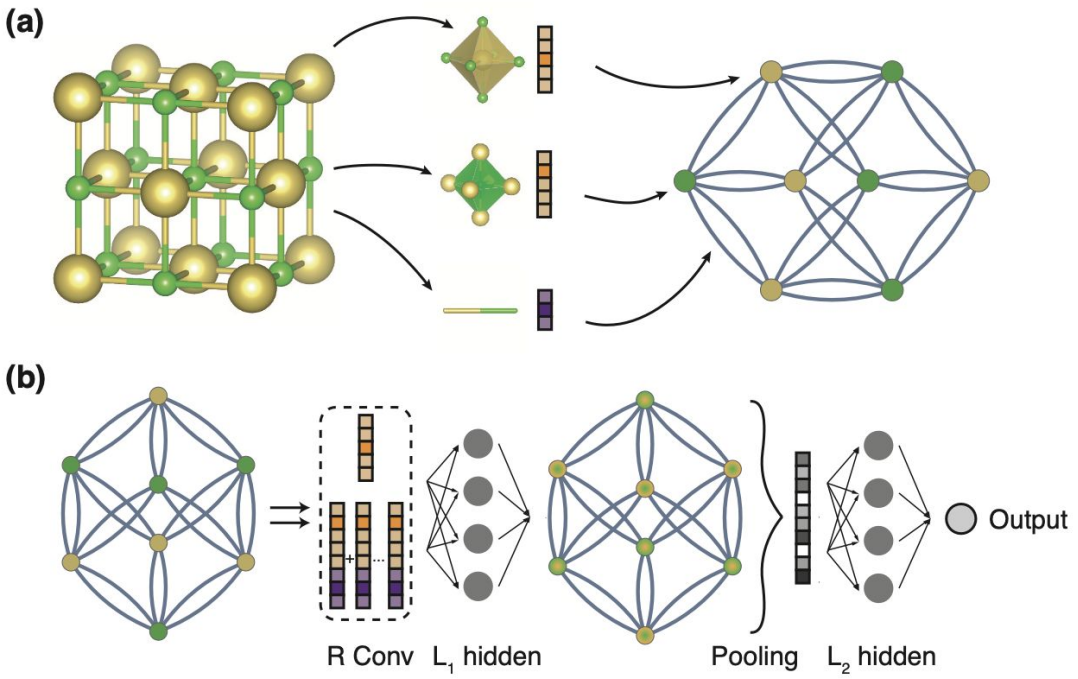
- Oversmoothing - use 2-4 message passing layers
- Expressivity limit - introduce higher order interactions (e.g., three-body terms)

Graph batching: how to train GNN for multiple structures?

Graphs cannot be stacked similarly to tabular data

- Several graphs are combined in a giant graph with multiple isolated components
- Since graphs are disconnected, message passing works as intended
- During post-processing, the component index is used to aggregate information from nodes that belong to individual graphs
- This allows efficient usage of resources and get one GNN for different crystals

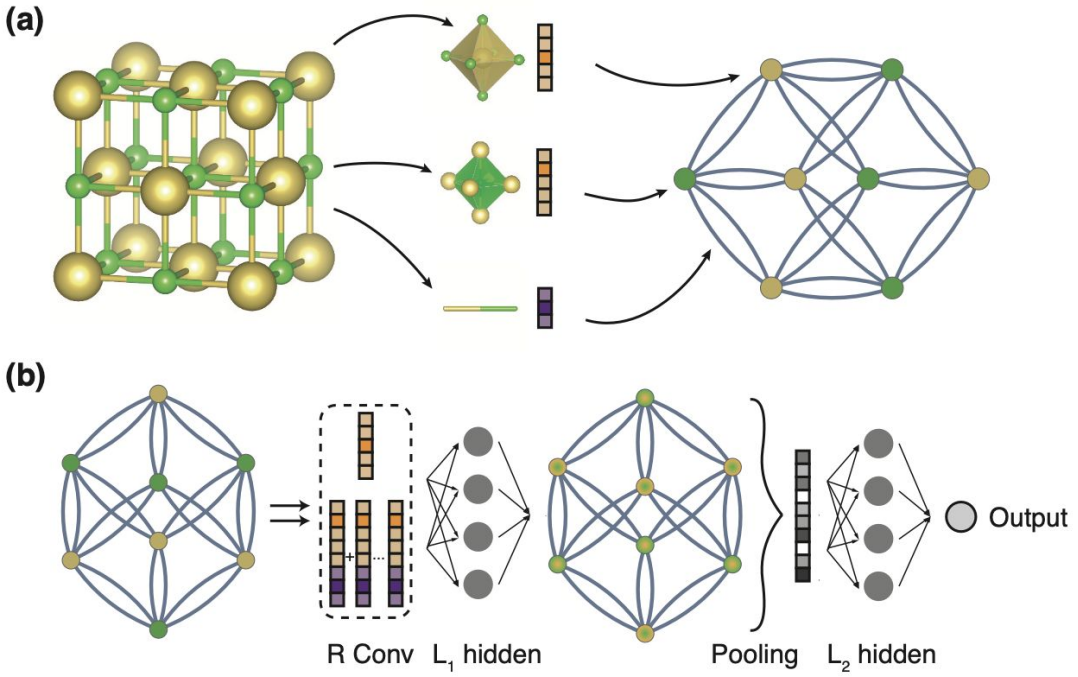
Example: Crystal graph convolutional neural network



Crystal Graph Convolutional Neural Networks for an Accurate and Interpretable Prediction of Material Properties

Pipeline

- Convert structure to graph
- Update node states (convolution)
- Apply pooling
- Apply post-processing layer (regression of classification)



Pipeline

- Convert structure to graph
- Update node states (convolution)
- Apply pooling
- Apply post-processing layer (regression of classification)

$$\boldsymbol{v}_i^{(t+1)} = \text{Conv}\left(\boldsymbol{v}_i^{(t)}, \boldsymbol{v}_j^{(t)}, \boldsymbol{u}_{(i,j)_k}\right), (i,j)_k \in \mathcal{G}$$

$$\boldsymbol{v_c} = \text{Pool}\left(\boldsymbol{v}_0^{(0)}, \boldsymbol{v}_1^{(0)}, ..., \boldsymbol{v}_N^{(0)}, ..., \boldsymbol{v}_N^{(R)}\right)$$

v_i is atom feature vector, $u(i,j)$ is bond feature vector, k - correspond to periodicity, t - passing layer number

Pipeline

- Convert structure to graph
- Update node states
- Apply pooling
- Apply post-processing layer (regression of classification)

$$\boldsymbol{z}_{(i,j)_k}^{(t)} = \boldsymbol{v}_i^{(t)} \oplus \boldsymbol{v}_j^{(t)} \oplus \boldsymbol{u}_{(i,j)_k}$$

$$\boldsymbol{v}_i^{(t+1)} = \boldsymbol{v}_i^{(t)} + \sum_{j,k} \sigma(\boldsymbol{z}_{(i,j)_k}^{(t)} \boldsymbol{W}_f^{(t)} + \boldsymbol{b}_f^{(t)}) \odot g(\boldsymbol{z}_{(i,j)_k}^{(t)} \boldsymbol{W}_s^{(t)} + \boldsymbol{b}_s^{(t)})$$

$$\boldsymbol{v}_c = \mathrm{Pool}\left(\boldsymbol{v}_0^{(0)}, \boldsymbol{v}_1^{(0)}, ..., \boldsymbol{v}_N^{(0)}, ..., \boldsymbol{v}_N^{(R)}\right)$$

$\oplus$ concatenate operation (combine)

$\odot$ element-wise multiplication

TABLE II. Properties used in atom feature vector $\boldsymbol{v}_i$

Property	Unit	Range	# of categories
Group number	–	1,2, ..., 18	18
Period number	–	1,2, ..., 9[a]	9
Electronegativity [S32, S33]	–	0.5–4.0	10
Covalent radius [S34]	pm	25–250	10
Valence electrons	–	1, 2, ..., 12	12
First ionization energy [S35][b]	eV	1.3–3.3	10
Electron affinity [S36]	eV	-3–3.7	10
Block	–	s, p, d, f	4
Atomic volume[b]	cm^3/mol	1.5–4.3	10

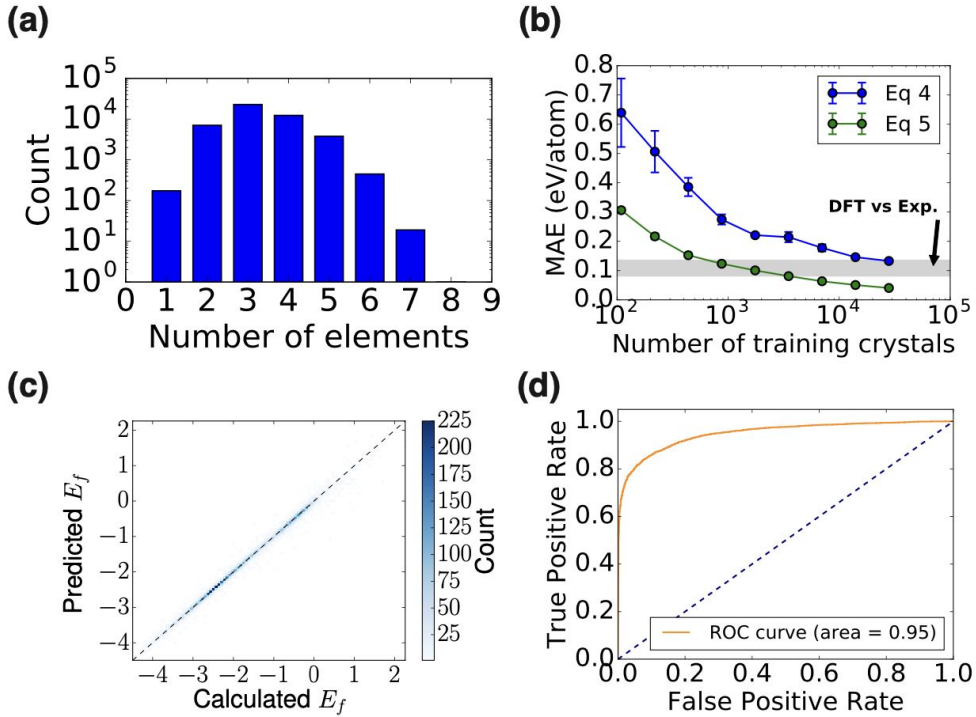
[a] The lanthanide and actinide elements are considered as period 8 and 9 respectively.
[b] Log scale is used for these properties.

TABLE III. Properties used in bond feature vector $\boldsymbol{u}_{(i,j)_k}$

Property	Unit	Range	# of categories
Atom distance	Å	0.7–5.2	10

SM: https://arxiv.org/pdf/1710.10324

Property	# of train data	Unit	MAE_{model}	MAE_{DFT}
Formation energy	28046	eV/atom	0.039	0.081–0.136[18]
Absolute energy	28046	eV/atom	0.072	–
Band gap	16458	eV	0.388	0.6[23]
Fermi energy	28046	eV	0.363	–
Bulk moduli	2041	log(GPa)	0.054	0.050[24]
Shear moduli	2041	log(GPa)	0.087	0.069[24]
Poisson ratio	2041	–	0.030	–



Evolution of Graph Neural Networks in Materials Science (2018–2026)

2018 — CGCNN: A Foundational Breakthrough
Crystal graphs for materials property prediction.

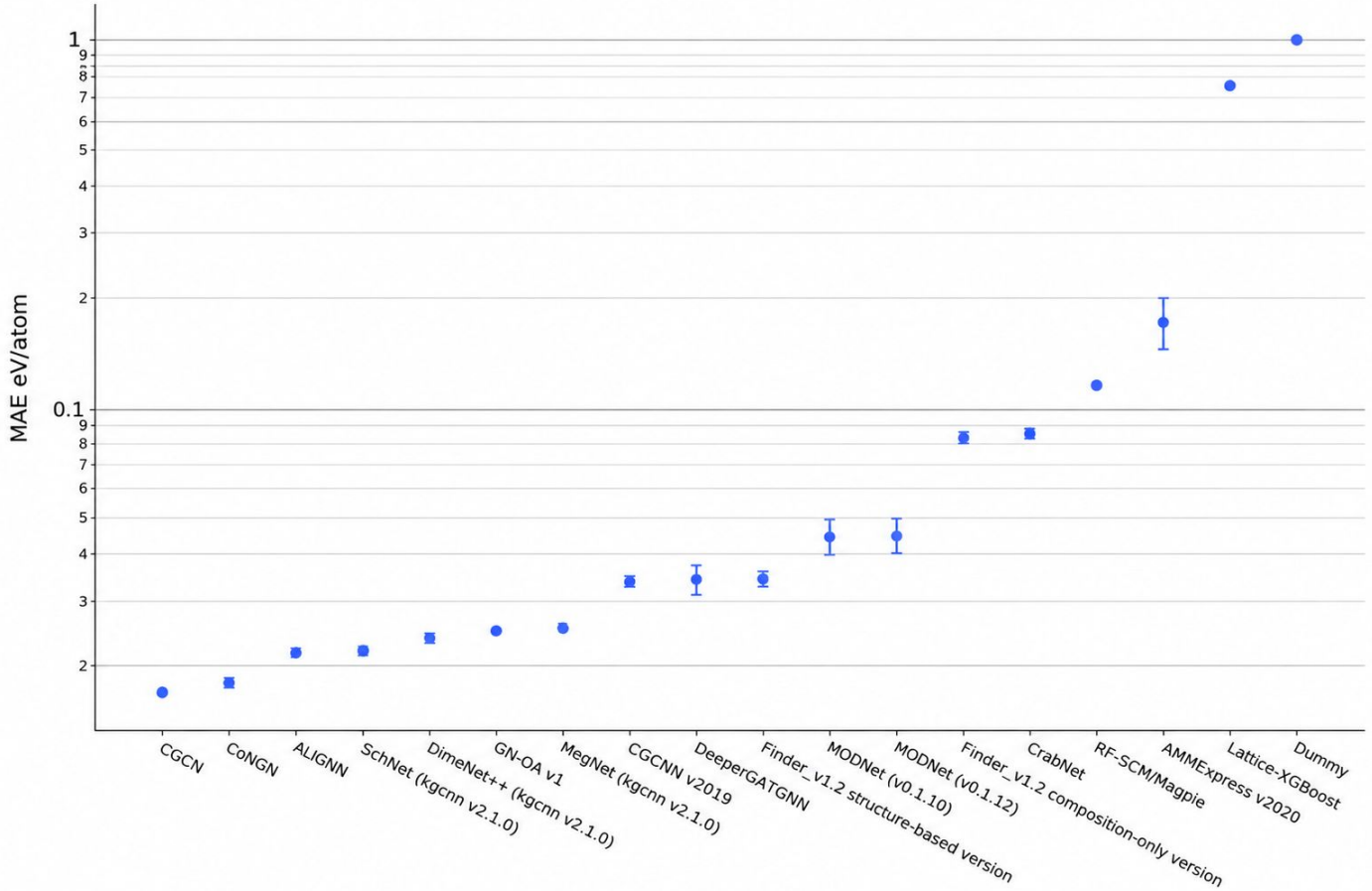
2018–2019 — SchNet, MEGNet
Continuous-filter convolutions, learnable atomic embeddings, and global state representations.

2020–2021 — DimeNet, GemNet, ALIGNN
Bond angles, directional message passing, and higher-order geometric interactions.

2021–2022 — NequIP, Allegro, MACE
E(3)-equivariant neural networks and accurate machine-learning interatomic potentials.

2023–2025 — CHGNet, MACE-MP, MatterSim, Orb
Universal pretrained interatomic potentials across diverse chemical systems.

2025–2026 — Foundation Models & Autonomous Discovery
Large-scale pretraining, fine-tuning, accelerated molecular dynamics, generative models, and closed-loop materials discovery.



Keep also in mind:

- cost
- specific properties
- transferability

Take home message

- Graph is an intuitive representation of an atomistic structure
- GNNs is a powerful tool for predicting materials properties
- Be aware of GNNs limitations

Literature

- Machine learning in materials informatics: recent applications and prospects
- DScribe: Library of descriptors for machine learning in materials science
- Crystal Graph Convolutional Neural Networks (arxiv) for an Accurate and Interpretable Prediction of Material Properties
- https://matbench-discovery.materialsproject.org/